

Logical Reasoning for Computer Science

COMPSCI 2LC3

McMaster University, Fall 2026

Wolfram Kahl

2026-09-09

LLMs can do Logical Reasoning? — o1 Example

Denali and Nate work for a dog walking business and are paid for each dog they walk. Denali is responsible for 16 dogs and Nate is responsible for 12 dogs. Under the company's new policy, they will be assigned or unassigned new dogs in groups of x dogs. The ratio of Denali's pay to Nate's pay would be the same if Denali started walking $4x$ more dogs and Nate stayed at 12 dogs or if x of Nate's dogs were reassigned to Denali. Find x if $x \neq 0$.

I need to find a value of x that makes the ratio of Denali's pay to Nate's pay the same in two different scenarios.

I can assume that the pay per dog is constant, so the ratio of their pay is the same as the ratio of the number of dogs they walk.

Let me write an equation for the first scenario, where Denali walks $4x$ more dogs and Nate stays at 12 dogs.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+4x}{12}$.

Now let me write an equation for the second scenario, where x of Nate's dogs are reassigned to Denali.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+x}{12-x}$.

I want these two ratios to be equal, so I can set them equal to each other and solve for x .

I get $\frac{16+4x}{12} = \frac{16+x}{12-x}$.

I can cross-multiply and simplify this equation.

I get $(16+4x)(12-x) = (16+x)(12)$.

Expanding the brackets, I get $192 - 16x + 48x - 4x^2 = 192 + 12x$.

Subtracting 192 from both sides, I get $-16x + 48x - 4x^2 = 12x$.

Rearranging the terms, I get $-4x^2 + 20x = 12x$.

Subtracting $12x$ from both sides, I get $-4x^2 + 8x = 0$.

I can factor out $-4x$ from both terms, and get $-4x(x-2) = 0$.

This means that either $-4x = 0$ or $x-2 = 0$.

Solving for x , I get $x = 0$ or $x = 2$.

However, the problem says that $x \neq 0$, so I can ignore that solution.

Therefore, the only valid solution is $x = 2$.

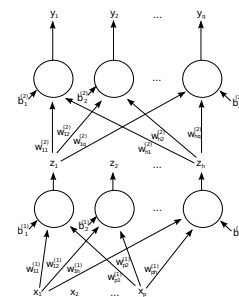
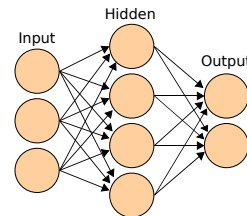
Answer: 2

What is “Generative AI”? — Artificial Neural Networks

Currently, “Large Language Models” and “Foundation Models” are **artificial neural networks** (ANNs).

- An ANN is a directed graph with:
 - designated tuples of input and output nodes
 - (usually with a (mostly-)layered organisation)
 - “activation functions” as node labels
 - floating-point or few-bit integer “weights” as edge labels.
- The edge weights are also called “parameters”. (GPT-4 is estimated to have over 10^{12} parameters.)
- An ANN calculates output node values from input node values by propagating node values along edges, and applying the activation function to the weighted sum of the incoming node values in each node with incoming edges. (This is frequently called “inference”).
- Determining the parameters of an ANN is called “training” or “learning”.

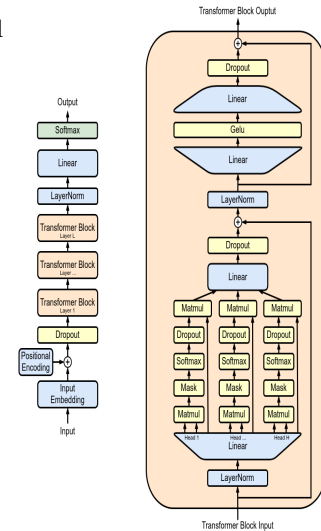
W: “Most learning models can be viewed as a straightforward application of optimization theory and statistical estimation.”



What is “Generative AI”? — “Transformers”

“Transformers”: The NN graph structure pattern currently-used in LLMs and “Foundation Models” (term coined in 2021)

- Acyclic graph (“only forward connections”, not “recurrent”)
- Therefore easier to “train” than cyclic “Recurrent Neural Networks” (including CNNs)
- “Residual edges” skip subnetworks
- “Attention” subnetworks can be understood as calculating “soft weights” used for emphasising certain input positions depending on context
- Geva *et al.* 2021: The feed-forward subnets of transformers effectively implement key-value stores
(The keys “are” sets of inputs (e.g., prefix sequences) mapped to the same output distribution.)
- (“The transformer paper” is from 2017; the essential idea was already published by Jürgen Schmidhuber in 1992.)

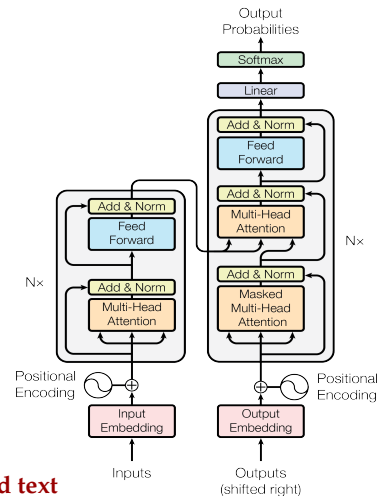


What is “Generative AI”? — “Generative Pretrained Transformers” (GPT) W

- LLMs are based on the transformer architecture.
- “Generative” in LLMs, simply speaking:
 - The input is a sequence of tokens
 - The output is **the next token**
 - The net is evaluated in a loop modifying the input by appending the previous-step output token at the end
- The “context window” limits
 - how much of the input influences the output
 - the length of the conversation used for the next answer
 - how much output could be generated (usually limited to smaller size)

Examples:

- Claude 2.1 (2023): 200k tokens context window
- GTP-4 Turbo (2024): 128k tokens context window, 4096 tokens output limit.
- LLMs are typically pre-trained on **large data sets of unlabelled text**
- LLMs are able to generate “**novel human-like content**”
- **Humans are fallible, and occasionally deceptive!**



o1 Example — What is the LLM Doing Here?

Denali and Nate work for a dog walking business and are paid for each dog they walk. Denali is responsible for 16 dogs and Nate is responsible for 12 dogs. Under the company's new policy, they will be assigned or unassigned new dogs in groups of x dogs. The ratio of Denali's pay to Nate's pay would be the same if Denali started walking $4x$ more dogs and Nate stayed at 12 dogs or if x of Nate's dogs were reassigned to Denali. Find x if $x \neq 0$.

I need to find a value of x that makes the ratio of Denali's pay to Nate's pay the same in two different scenarios.

I can assume that the pay per dog is constant, so the ratio of their pay is the same as the ratio of the number of dogs they walk.

Let me write an equation for the first scenario, where Denali walks $4x$ more dogs and Nate stays at 12 dogs.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+4x}{12}$.

Now let me write an equation for the second scenario, where x of Nate's dogs are reassigned to Denali.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+x}{12-x}$.

I want these two ratios to be equal, so I can set them equal to each other and solve for x .

I get $\frac{16+4x}{12} = \frac{16+x}{12-x}$.

I can cross-multiply and simplify this equation.

I get $(16+4x)(12-x) = (16+x)(12)$.

Expanding the brackets, I get $192 - 16x + 48x - 4x^2 = 192 + 12x$.

Subtracting 192 from both sides, I get $-16x + 48x - 4x^2 = 12x$.

Rearranging the terms, I get $-4x^2 + 20x = 12x$.

Subtracting $12x$ from both sides, I get $-4x^2 + 8x = 0$.

I can factor out $-4x$ from both terms, and get $-4x(x-2) = 0$.

This means that either $-4x = 0$ or $x-2 = 0$.

Solving for x , I get $x = 0$ or $x = 2$.

However, the problem says that $x \neq 0$, so I can ignore that solution.

Therefore, the only valid solution is $x = 2$.

Answer: 2

Formalisation: Natural language to mathematical notation

“Working” with mathematical notation

LLMs can do Logical Reasoning? — o1 Example, Formalisation Part

Denali and Nate work for a dog walking business and are paid for each dog they walk. Denali is responsible for 16 dogs and Nate is responsible for 12 dogs. Under the company's new policy, they will be assigned or unassigned new dogs in groups of x dogs. The ratio of Denali's pay to Nate's pay would be the same if Denali started walking $4x$ more dogs and Nate stayed at 12 dogs or if x of Nate's dogs were reassigned to Denali. Find x if $x \neq 0$.

I need to find a value of x that makes the ratio of Denali's pay to Nate's pay the same in two different scenarios.

I can assume that the pay per dog is constant, so the ratio of their pay is the same as the ratio of the number of dogs they walk.

Let me write an equation for the first scenario, where Denali walks $4x$ more dogs and Nate stays at 12 dogs.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+4x}{12}$.

Now let me write an equation for the second scenario, where x of Nate's dogs are reassigned to Denali.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+x}{12-x}$.

I want these two ratios to be equal, so I can set them equal to each other and solve for x .

I get $\frac{16+4x}{12} = \frac{16+x}{12-x}$.

At this point, a problem statement in a **formal mathematical language** has been reached.

o1 Example: Formal Part Done Informally, and Wrong

I get $\frac{16+4x}{12} = \frac{16+x}{12-x}$.

I can cross-multiply and simplify this equation.

I get $(16+4x)(12-x) = (16+x)(12)$.

Expanding the brackets, I get $192 - 16x + 48x - 4x^2 = 192 + 12x$.

Subtracting 192 from both sides, I get $-16x + 48x - 4x^2 = 12x$.

Rearranging the terms, I get $-4x^2 + 20x = 12x$.

Subtracting $12x$ from both sides, I get $-4x^2 + 8x = 0$.

I can factor out $-4x$ from both terms, and get $-4x(x-2) = 0$.

This means that either $-4x = 0$ or $x-2 = 0$.

Solving for x , I get $x = 0$ or $x = 2$.

However, the problem says that $x \neq 0$, so I can ignore that solution.

Therefore, the only valid solution is $x = 2$.

Answer: 2

LLMs can do Logical Reasoning? — o1 Example, Formalisation Part

Denali and Nate work for a dog walking business and are paid for each dog they walk. Denali is responsible for 16 dogs and Nate is responsible for 12 dogs. Under the company's new policy, they will be assigned or unassigned new dogs in groups of x dogs. The ratio of Denali's pay to Nate's pay would be the same if Denali started walking $4x$ more dogs and Nate stayed at 12 dogs or if x of Nate's dogs were reassigned to Denali. Find x if $x \neq 0$.

I need to find a value of x that makes the ratio of Denali's pay to Nate's pay the same in two different scenarios.

I can assume that the pay per dog is constant, so the ratio of their pay is the same as the ratio of the number of dogs they walk.

Let me write an equation for the first scenario, where Denali walks $4x$ more dogs and Nate stays at 12 dogs.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+4x}{12}$.

Now let me write an equation for the second scenario, where x of Nate's dogs are reassigned to Denali.

The ratio of Denali's dogs to Nate's dogs is $\frac{16+x}{12-x}$.

I want these two ratios to be equal, so I can set them equal to each other and solve for x .

I get $\frac{16+4x}{12} = \frac{16+x}{12-x}$.

A proper **neuro-symbolic system (NSS)** would now call a **symbolic solver** on:

$$x \neq 0 \quad \wedge \quad \frac{16+4x}{12} = \frac{16+x}{12-x}$$

Symbolic Solving? — Logical Reasoning means Creating Proofs

What does it mean to “symbolically solve $x \neq 0 \wedge \frac{16+4x}{12} = \frac{16+x}{12-x}$ ”?

- Recognize “ $x \neq 0 \wedge \frac{16+4x}{12} = \frac{16+x}{12-x}$ ” as a construct in a formal language of mathematics.
- Follow a well-defined process that applies syntax transformation rules in each step.
- The choice of rules will be driven by what we are solving for, here, “ $x = ?$ ”.
- Choosing appropriate rules to apply is in general not automatable – it **requires experience**.
- If a solution is found, the process will have created a **proof**.
- Proofs are again constructs in formal languages of mathematics.
- Appropriate systems of rules are provided by **logics**.
- Trustworthy logical reasoning is fundamentally different from neural-network “inference”.

Trustworthy Reasoning: Symbolic Reasoning

- **Symbolic reasoning:** Expression, formulae, and proofs are treated as elements of (typically tree-like) datatypes
- **Syntactic manipulation:** functions defined on the structure of these trees
- Syntactic manipulation itself does not depend on the **meaning** (semantics) of the expressions
- **Sound** syntactic manipulation preserves the semantics
- **Checking whether a proof follows the (syntactic) rules is mostly relatively easy to automate**
- Checking whether a formula is semantically valid is usually **undecidable**, i.e., cannot be fully automated

```
data Prop      -- Prop. logic expressions :
= Var String   -- variables
| Const Bool   -- constants `True` and `False`
| Not Prop     -- applications of unary operator `Not`
| And Prop Prop -- applications of binary operators
| Or Prop Prop  -- `And` respectively `Or`

propNNF :: Prop -> Prop -- returns negation normal form
propNNF (Var v) = Var v
propNNF (Const b) = Const (not b)
propNNF (Not p) = neg p
  where
    neg :: Prop -> Prop -- returns NNF of negation
    neg (Var v) = Not (Var v)
    neg (Not p) = propNNF p
    neg (And p1 p2) = Or (neg p1) (neg p2)
    neg (Or p1 p2) = And (neg p1) (neg p2)
propNNF (And p1 p2) = And (propNNF p1) (propNNF p2)
propNNF (Or p1 p2) = Or (propNNF p1) (propNNF p2)
```

What is This Course About? Logical Reasoning: “How to Not Hallucinate ...”

- Calendar description:

Introduction to logic and proof techniques for practical reasoning: propositional logic, predicate logic, structural induction; rigorous proofs in discrete mathematics and programming.

- **Logical Reasoning** is used for
 - **proving theorems in all kinds of application areas**
 - **justifying software designs**
 - **proving sophisticated algorithms correct**
 - **proving software implementations correct**
- **Discrete Mathematics** is
 - the math of data and **data structures**
 - the math of reasoning—**logic**
 - the math of some kinds of AI—**symbolic machine reasoning**
 - the math of **specifying software**
 - **a good application area for practicing logical reasoning in CS**

Goals and Rough Outline

- Understand the mechanics of mathematical expressions and proof
— starting in a familiar area: **Reasoning about integers**
- Develop skill in **propositional calculus**
 - “**propositional**”: statements that can be true or false, not numbers
 - “**calculus**”: **formalised** reasoning, **calculation** — $\mathbb{B}, \neg, \wedge, \vee, \Rightarrow, \dots$
- Develop skill in **predicate calculus** — $\forall, \exists$
- Develop skill in using **basic theories of “data mathematics”**
 - Sets, Functions, **Relations**, Sequences, Trees, Graphs, ...
 - Correctness of functional programs
- Develop skill in **correctness reasoning about (imperative) programs**
- ... *skill development takes time and effort* ...

All along:

- Encounter computer support for logical reasoning, mechanised discrete mathematics
- Introduction to mechanised software correctness tools
— **Formal Methods**: increasingly important in industry

A Formal Calculation — (see Homework 1, due Friday, 14:30)

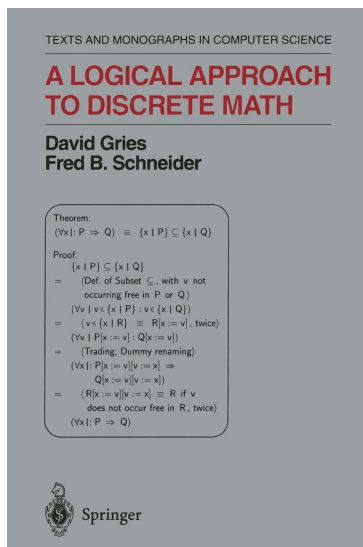
Calculation: (starting from a division-free formalisation of the o1 Example)

Assuming $x \neq 0$:

```
(16 + 4 · x) · (12 - x) = (16 + x) · 12
≡( "Distributivity of · over -" )
(16 + 4 · x) · 12 - (16 + 4 · x) · x = (16 + x) · 12
≡( "Distributivity of · over +" )
16 · 12 + 4 · x · 12 - (16 · x + 4 · x · x) = 16 · 12 + x · 12
≡( "Mutual associativity of + and -" )
16 · 12 + (4 · x · 12 - (16 · x + 4 · x · x)) = 16 · 12 + x · 12
≡( "Cancellation of +" )
4 · 12 · x - (16 · x + 4 · x · x) = 12 · x
≡( "Subtraction of addition" )
4 · 12 · x - 16 · x - 4 · x · x = 12 · x
≡( "Distributivity of · over -" )
(4 · 12 - 16 - 4 · x) · x = 12 · x
≡( "Cancellation of ·" with assumption `x ≠ 0` )
4 · 12 - 16 - 4 · x = 12
≡( Fact `4 · 12 - 16 = 12 + 20` )
12 + 20 - 4 · x = 12
≡( "Subtraction" )
```

...

Textbook: “LADM”



“This is a rather extraordinary book, and deserves to be read by everyone involved in computer science and — perhaps more importantly — software engineering. I recommend it highly [...]. If the book is taken seriously, the rigor that it unfolds and the clarity of its concepts could have a significant impact on the way in which software is conceived and developed.”

— Peter G. Neumann
(Founder of ACM SIGSOFT)

Calculational Proof Format — Equational Calculations

$$\begin{array}{l} E_0 \\ = \langle \text{Explanation of why } E_0 = E_1 \rangle \\ E_1 \\ = \langle \text{Explanation of why } E_1 = E_2 \rangle \\ E_2 \\ = \langle \text{Explanation of why } E_2 = E_3 \rangle \\ E_3 \end{array}$$

This reads as:

$$E_0 = E_1 \quad \wedge \quad E_1 = E_2 \quad \wedge \quad E_2 = E_3$$

Because $=$ is **transitive**, this justifies:

$$E_0 = E_3$$

Calculational Proof Format — Generalised

$$\begin{array}{l} E_0 \\ < \langle \text{Explanation of why } E_0 < E_1 \rangle \\ E_1 \\ = \langle \text{Explanation of why } E_1 = E_2 \rangle \\ E_2 \\ \leq \langle \text{Explanation of why } E_2 \leq E_3 \rangle \\ E_3 \end{array}$$

This reads as:

$$E_0 < E_1 \quad \wedge \quad E_1 = E_2 \quad \wedge \quad E_2 \leq E_3$$

Because we have appropriate variants of transitivity laws for $<$, $=$, and $\leq$, this justifies:

$$E_0 < E_3$$

Calculational Non-Proofs

$$\begin{array}{l} E_0 \\ \leq \langle \text{Explanation of why } E_0 \leq E_1 \rangle \\ E_1 \\ = \langle \text{Explanation of why } E_1 = E_2 \rangle \\ E_2 \\ \geq \langle \text{Explanation of why } E_2 \geq E_3 \rangle \\ E_3 \end{array}$$

This reads as:

$$E_0 \leq E_1 \quad \wedge \quad E_1 = E_2 \quad \wedge \quad E_2 \geq E_3$$

This justifies nothing about the relation between E_0 and E_3 !

An Equational Theory of Integers — Axioms (LADM Ch. 15)

- (15.1) **Axiom, Associativity:** $(a + b) + c = a + (b + c)$
 $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- (15.2) **Axiom, Symmetry:** $a + b = b + a$
 $a \cdot b = b \cdot a$
- (15.3) **Axiom, Additive identity:** $0 + a = a$
 $a + 0 = a$
- (15.4) **Axiom, Multiplicative identity:** $1 \cdot a = a$
 $a \cdot 1 = a$
- (15.5) **Axiom, Distributivity:** $a \cdot (b + c) = a \cdot b + a \cdot c$
 $(b + c) \cdot a = b \cdot a + c \cdot a$
- (15.13) **Axiom, Unary minus:** $a + (-a) = 0$
- (15.14) **Axiom, Subtraction:** $a - b = a + (-b)$

Calculational Proofs of Theorems — (15.17) $-(-a) = a$

(15.3) Identity of + $0 + a = a$	(15.13) Unary minus $a + (-a) = 0$
----------------------------------	------------------------------------

LADM:

Theorem (15.17): $-(-a) = a$

Proof:

$$\begin{aligned}
 & -(-a) \\
 = & \langle \text{Identity of + (15.3)} \rangle \\
 & 0 + -(-a) \\
 = & \langle \text{Unary minus (15.13)} \rangle \\
 & a + (-a) + -(-a) \\
 = & \langle \text{Unary minus (15.13)} \rangle \\
 & a + 0 \\
 = & \langle \text{Identity of + (15.3)} \rangle \\
 & a
 \end{aligned}$$

Calculational Proofs of Theorems — (15.17) $-(-a) = a$

(15.3) Identity of + $0 + a = a$	(15.13) Unary minus $a + (-a) = 0$
----------------------------------	------------------------------------

LADM:

Theorem (15.17): $-(-a) = a$

Proof:

$$\begin{aligned}
 & -(-a) \\
 = & \langle \text{Identity of + (15.3)} \rangle \\
 & 0 + -(-a) \\
 = & \langle \text{Unary minus (15.13)} \rangle \\
 & a + (-a) + -(-a) \\
 = & \langle \text{Unary minus (15.13)} \rangle \\
 & a + 0 \\
 = & \langle \text{Identity of + (15.3)} \rangle \\
 & a
 \end{aligned}$$

CALC CHECK:

Theorem (15.17) “Self-inverse of unary minus”:

$$-(-a) = a$$

Proof:

$$\begin{aligned}
 & -(-a) \\
 = & \langle \text{“Identity of +”} \rangle \\
 & 0 + -(-a) \\
 = & \langle \text{“Unary minus”} \rangle \\
 & a + (-a) + -(-a) \\
 = & \langle \text{“Unary minus”} \rangle \\
 & a + 0 \\
 = & \langle \text{“Identity of +”} \rangle \\
 & a
 \end{aligned}$$

in Ex1.2

First Tool: CALCCHECK

- CALCCHECK: A proof checker for the logic of LADM
 - CALCCHECK analyses LADM-style presentations of proofs
 - CALCCHECK_{Web}: A notebook-style web-app interface to CALCCHECK
 - **You can check your proofs before handing them in!**
 - **Will be used in exams!**
 - with syntax checking always left on,
 - but initially with proof checking turned off. . .
 - **Will be used in exams**
 - as far as possible. . .
- You need to be able to do both:**
- Write formalisations and proofs using CALCCHECK
 - **Write formalisations and proofs by hand on paper**
- (SAS accommodations specifying extra time are normally accommodated in the labs.)
 - (Firefox and Chrome can be expected to work with CALCCHECK_{Web}.)

CALCCHECK: A Recognisable Version of the LADM Proof Language

(11.5) $S = \{x \mid x \in S : x\}$.

According to axiom Extensionality (11.4), it suffices to prove that $v \in S \equiv v \in \{x \mid x \in S : x\}$, for arbitrary v . We have,

$$\begin{aligned} & v \in \{x \mid x \in S : x\} \\ = & \quad \langle \text{Definition of membership (11.3)} \rangle \\ & (\exists x \mid x \in S : v = x) \\ = & \quad \langle \text{Trading (9.19), twice} \rangle \\ & (\exists x \mid x = v : x \in S) \\ = & \quad \langle \text{One-point rule (8.14)} \rangle \\ & v \in S \end{aligned}$$

Theorem (11.5): $S = \{x \mid x \in S : x\}$
 Proof:
 Using "Set extensionality" (11.4):
 For any `v`:

$$\begin{aligned} & v \in \{x \mid x \in S : x\} \\ = & \quad \langle \text{"Set membership" (11.3)} \rangle \\ & (\exists x \mid x \in S : v = x) \\ = & \quad \langle \text{"Trading for } \exists \text{" (9.19)} \rangle \\ & (\exists x \mid x = v : x \in S) \\ = & \quad \langle \text{"One-point rule for } \exists \text{" (8.14), substitution} \rangle \\ & v \in S \end{aligned}$$

Note:

1. The calculation part is transliterated into Unicode plain text (only minimal notation changes).
2. The prose top-level of the proof is formalised into Using and For any structures in the spirit of LADM

An Equational Theory of Integers — Axioms (CALCCHECK)

Declaration: $\mathbb{Z}$: Type

Declaration: $_+ : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \mathbb{Z}$

Declaration: $_{\cdot} : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \mathbb{Z}$

Axiom (15.1) (15.1a) "Associativity of $+$ ": $(a + b) + c = a + (b + c)$

Axiom (15.1) (15.1b) "Associativity of $\cdot$ ": $(a \cdot b) \cdot c = a \cdot (b \cdot c)$

Axiom (15.2) (15.2a) "Symmetry of $+$ ": $a + b = b + a$

Axiom (15.2) (15.2b) "Symmetry of $\cdot$ ": $a \cdot b = b \cdot a$

Axiom (15.3) "Additive identity" "Identity of $+$ ": $0 + a = a$

Axiom (15.4) "Multiplicative identity" "Identity of $\cdot$ ": $1 \cdot a = a$

Axiom (15.5) "Distributivity of $\cdot$ over $+$ ": $a \cdot (b + c) = a \cdot b + a \cdot c$

Axiom (15.9) "Zero of $\cdot$ ": $a \cdot 0 = 0$

Declaration: $_- : \mathbb{Z} \rightarrow \mathbb{Z}$

Declaration: $_{-} : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \mathbb{Z}$

Axiom (15.13) "Unary minus": $a + (-a) = 0$

Axiom (15.14) "Subtraction": $a - b = a + (-b)$

Ex1.2 Starting Point

Why is “ $2 \cdot 3 + 4$ ” an expression?

- If $\otimes$ is a **binary infix operator** and D and E are expressions, then $D \otimes E$ is an expression, with operands D and E .

An expression can be ...

- or the application of **some binary infix operator** to **two simpler expressions**

Which of the following shows the (tree) structure of “ $2 \cdot 3 + 4$ ”?



Why?

$\Rightarrow$ The multiplication operator $\cdot$ has **higher precedence** than the addition operator $+$.

LADM Table of Precedences

- $[x := e]$ (textual substitution) (highest precedence)
- $\cdot$ (function application)
- unary prefix operators $+$, $-$, $\neg$, $\#$, $\sim$, $\mathcal{P}$
- $**$
- $\cdot$ / $\div$ mod gcd
- $+$ $-$ $\cup$ $\cap$ $\times$ $\circ$ $\bullet$
- $\downarrow$ $\uparrow$
- $\#$
- $\triangleleft$ $\triangleright$ $\wedge$
- $=$ $\neq$ $<$ $>$ $\in$ $\subset$ $\subseteq$ $\supset$ $\supseteq$ $|$ (conjunctive)
- $\vee$ $\wedge$
- $\Rightarrow$ $\nRightarrow$ $\Leftarrow$ $\nLeftarrow$
- $\equiv$ $\neq$ (lowest precedence)

All non-associative binary infix operators **associate to the left**, except $**$, $\triangleleft$, $\Rightarrow$, $\rightarrow$, which **associate to the right**.

Associativity versus Association

- If we write $a + b + c$, there appears to be no need to discuss whether we mean $(a + b) + c$ or $a + (b + c)$, because they evaluate to the same values:

$$(a + b) + c = a + (b + c) \quad \text{"+" is associative}$$

- If we write $a - b - c$, we mean $(a - b) - c$:

$$\text{"-" associates to the left} \quad 9 - (5 - 2) \neq (9 - 5) - 2$$

- If we write a^{b^c} , we mean $a^{(b^c)}$:

$$\text{exponentiation associates to the right} \quad 2^{(3^2)} \neq (2^3)^2$$

- If we write $a ** b ** c$, we mean $a ** (b ** c)$:

$$\text{"**" associates to the right}$$

- If we write $a \Rightarrow b \Rightarrow c$, we mean $a \Rightarrow (b \Rightarrow c)$:

$$\text{"}\Rightarrow\text{" associates to the right} \quad (F \Rightarrow (T \Rightarrow F)) \neq ((F \Rightarrow T) \Rightarrow F)$$

- If we write $a \rightarrow b \rightarrow c$, we mean $a \rightarrow (b \rightarrow c)$:

The function type constructor “ $\rightarrow$ ” associates to the right $(\mathbb{Z} \rightarrow (\mathbb{Z} \rightarrow \mathbb{B})) \neq ((\mathbb{Z} \rightarrow \mathbb{Z}) \rightarrow \mathbb{B})$

Automatic Application of Associativity and Symmetry Laws

Axiom (15.1) (15.1a) "Associativity of +": $(a + b) + c = a + (b + c)$

Axiom (15.1) (15.1b) "Associativity of ·": $(a \cdot b) \cdot c = a \cdot (b \cdot c)$

Axiom (15.2) (15.2a) "Symmetry of +": $a + b = b + a$

Axiom (15.2) (15.2b) "Symmetry of ·": $a \cdot b = b \cdot a$

- You have been trained to reason "up to symmetry and associativity"
- Making symmetry and associativity steps explicit is
 - **always allowed**
 - sometimes **very useful for readability**
- CALCCHECK allows selective activation of symmetry and associativity laws
 - ⇒ "Exercise ... / Assignment ...: [...] **without automatic associativity and symmetry**"
 - ⇒ Having to make symmetry and associativity steps explicit can be tedious...

Substitution is Useful! (and will be revisited as well...)

- **Substitution as such:** Replaces variables with expressions in expressions, e.g.,

$$\begin{aligned} & (x + 2 \cdot y)[x, y := 3 \cdot a, b + 5] \\ &= \langle \text{Substitution} \rangle \\ & 3 \cdot a + 2 \cdot (b + 5) \end{aligned}$$

- **Applying substitution instances of theorems** and making the substitution explicit:

$$\begin{aligned} & 2 \cdot y + - (2 \cdot y) \\ &= \langle \text{"Unary minus" } a + -a = 0 \text{ with } a := 2 \cdot y \rangle \\ & 0 \end{aligned}$$

- Proving correctness of assignment commands: $Q[x := E] \Rightarrow [x := E] Q$

Example Precondition-Postcondition Correctness Statements

- A **command correctness statement** in dynamic logic notation (as used in CALCCHECK)

$$P \Rightarrow [C] Q$$

means:

"If command C is started in a state in which the **precondition** P holds, then it will terminate only in a state in which the **postcondition** Q holds."

- Examples:

- $(x = 0) \Rightarrow [x := x + 1] (x = 1)$ ✓
- $(x < 0) \Rightarrow [x := x + 1] (x > 0)$ ✗
- $(x > 0) \Rightarrow [x := x + 1] (x > 0)$ ✓ (if $x : \mathbb{Z}$)

Correctness of Assignment Commands

- The Axiom “Assignment”: $Q[x := E] \Rightarrow [x := E] Q$ captures the following:

“If the command “ $x := E$ ” is expected to establish the **postcondition** Q , then it needs to be started in a state in which the **precondition** $Q[x := E]$ holds.”

- The dynamic-logic notation allows for calculational presentation:

$$\begin{aligned} & x + 1 = 5 \\ \equiv & \quad \quad \quad \langle \text{Substitution} \rangle \\ & (x = 5) [x := x + 1] \\ \Rightarrow & [x := x + 1] \quad \langle \text{“Assignment”} \rangle \\ & x = 5 \end{aligned}$$

- Substitution “:=”:**
One Unicode character; type “\ :=”

- Assignment “:=”:**
Two characters; type “:=”

Example Proof for a Sequence of Assignments

Lemma (4):

$$\begin{aligned} & x = 5 \\ \Rightarrow & [y := x + 1; \\ & \quad x := y + y \\ & \quad] \\ & x = 12 \end{aligned}$$

**Read and write
such “ $\Rightarrow$ ” proofs
from the bottom to the top!**

Proof:

$$\begin{aligned} & x = 5 \\ \equiv & \langle \text{“Cancellation of +”} \rangle \\ & x + 1 = 5 + 1 \\ \equiv & \langle \text{Fact `5 + 1 = 6`} \rangle \\ & x + 1 = 6 \\ \equiv & \langle \text{Substitution} \rangle \\ & (y = 6)[y := x + 1] \\ \Rightarrow & [y := x + 1] \quad \langle \text{“Assignment”} \rangle \\ & y = 6 \\ \equiv & \langle \text{“Cancellation of `·’ with Fact `2 ≠ 0’} \rangle \\ & 2 \cdot y = 2 \cdot 6 \\ \equiv & \langle \text{Evaluation} \rangle \\ & (1 + 1) \cdot y = 12 \\ \equiv & \langle \text{“Distributivity of `·’ over +”} \rangle \\ & 1 \cdot y + 1 \cdot y = 12 \\ \equiv & \langle \text{“Identity of `·’} \rangle \\ & y + y = 12 \\ \equiv & \langle \text{Substitution} \rangle \\ & (x = 12)[x := y + y] \\ \Rightarrow & [x := y + y] \quad \langle \text{“Assignment”} \rangle \\ & x = 12 \end{aligned}$$

COMPSCI 2LC3 Organisation

- Schedule
- Grading
- Exams
- Avenue
- Course Page: <http://www.cas.mcmaster.ca/~kahl/CS2LC3/2026/>
— check in case of Avenue outage!

— See the Outline (on course page and on Avenue)

- Slides — will not cover the complete lectures, but will be posted on Avenue, usually after the lecture.

Schedule

	Mon	Tue	Wed	Thu	Fri
8:30– –10:20		T5			
10:30– –12:20	T1, T6				
13:30–14:20					Instructor Office Hour
14:30–15:20	Lecture		Lecture		
16:30–17:20			T2, T3, T4		Lecture
17:30–18:20			T2, T3, T4		

- **Lectures: Attend! Take notes!**
- **2-hour Tutorials** — **starting today, September 9**
 - discuss student approaches to “Exercise” questions.
- **TA office hours: TBA**
- **Studying and Homework:** — **Reading for understanding and remembering**
 - **Writing proofs in CALCCHECK_{Web}**

Grading

- **Homework**, from one lecture to the next — in total: **0%**
 - Homework will be more frequent in the first part of the term
- **Roughly-biweekly Assignments** — in total: **0%**
 - Assignments will be less frequent in the first part of the term
- **Quizzes**, closed book, **on CALCCHECK_{Web} / on paper** — in total: **8%**
 - Short quizzes are conducted in some tutorial sessions
 - The weakest 1 or 2 quizzes are dropped (see outline)
 - MSAFs for homework are not processed
- **2 Midterm Tests**, closed book, **on CALCCHECK_{Web} / on paper**, each:
 - **15%** if not better than your final
 - **25%** if better than your final

— in total at least: **30%**
 — in total up to: **50%**
- **Final** (closed book, 2.5 hours, **on CALCCHECK_{Web} / on paper**) **42% to 62%**

= 100%

What to Expect in Exams

- Exercise questions, assignment questions, and the questions on midterm tests, and on the final —
 - **will be somewhat similar...**
- All tests and exams are **closed-book**.
 - The main difference to open-book lies in how you prepare...
 - **Knowledge is important:**
 - Without the right knowledge, you would not even know what to look up where!
- **You need to be able and prepared to do both:**
 - Write formalisations and proofs using CALCCHECK
 - Write formalisations and proofs by hand on paper
- **Know your stuff!**
 - ... and not only in the exams ...
 - ... **and not only for this term...**
 - ... **similar to learning a new language**

The Language of Logical Reasoning

The mathematical foundations of Computing Science involve **language skills and knowledge**:

- **Vocabulary**: Commonly known concepts and technical terms
- **Syntax/Grammar**: How to produce complex statements and arguments
- **Semantics**: How to relate complex statements with their meaning
- **Pragmatics**: How people actually use the features of the language

Conscious and fluent use of the
language of logical reasoning
is the foundation for
precise specification and rigorous argumentation
in **Computer Science and Software Engineering**.